



setTimeout vs await 이해하기

비동기 예약과 '진짜 기다림'의 차이를 7단계로 완벽하게 학습합니다.



동작 원리 이해

setTimeout이 왜 기다리지 않는지 실제 동작 메커니즘을 파헤칩니다.



진짜 기다림 구현

Promise와 await를 사용하여 실행 흐름을 제어하는 방법을 배웁니다.



올바른 패턴

흔한 오해를 바로잡고 실무에서 사용하는 정확한 코드 패턴을 익힙니다.

</> 코드 예제

index.js

```
1 console.log('A')
2 setTimeout(() => console.log('B'), 3000)
3 console.log('C')
```



💡 `setTimeout`은 실행을 잠시 멈추는 것이 아니라, "나중에 실행해줘"라고 브라우저에게 부탁(예약)하고 바로 다음 줄로 넘어갑니다.

× 대표 오해

"setTimeout(3000)은 3초 기다린다"

→ 틀린 말입니다. 코드 실행은 멈추지 않습니다.

1.2 실제 출력 순서

Non-blocking Model

자바스크립트는 단일 스레드(Single Thread) 언어입니다. 한 번에 하나의 작업만 처리할 수 있기 때문에, 오래 걸리는 작업(타이머 등)을 만나면 기다리지 않고 Web API에게 맡긴 뒤 바로 다음 일을 처리합니다.

blocking-test.js

```
// 1. 타이머 예약 (즉시 리턴됨)
setTimeout(()=>{}, 3000)

// 2. 바로 실행됨 (멈추지 않음!)
console.log("다음줄")
```

CONSOLE OUTPUT

“다음은”

— 딜레이 없이 즉시 출력됨

i **블로킹(Blocking) 하지 않음**

setTimeout은 실행을 멈추는 함수가 아니라, "등록"하는 함수입니다.

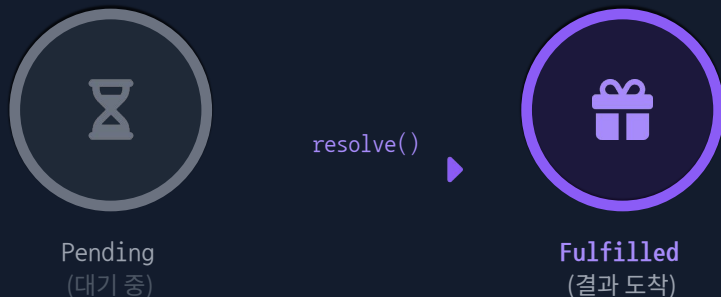
"자바스크립트는 기다려주지 않습니다."

Promise 객체의 역할

자바스크립트에서 "기다렸다가 결과를 주는" 방식을 만들기 위해 존재하는 특수한 객체입니다.

단순히 타이머만 돌리는 것이 아니라, 작업의 성공 또는 실패라는 상태를 관리하고 전달합니다.

Promise Concept Visualization



“ 언젠가 완료되면, 약속한 값을 열어볼 수 있어! ”

Promise의 두 가지 약속

1

미래에 결과를 넘겨준다

작업이 끝나면 성공 값(resolve)이나 실패 이유(reject)를 반드시 전달합니다.

2

기다리는 문법이 있다

결과가 준비될 때까지 코드를 멈추고 기다릴 수 있는 키워드 `await`를 사용할 수 있습니다.

promise_basic.js

```
const myPromise = new Promise((resolve) => {  
  // 나중에 완료되면 resolve 호출  
  setTimeout(() => resolve("완료!"), 3000)  
})
```

```
await myPromise // ⌚ 여기서 진짜 기다림!
```

</> wait 함수 만들기

utils.js

```
1 const wait = (ms) => new Promise(resolve => {  
2   // ms 밀리초 후에 resolve를 호출하여 약속 이행  
3   setTimeout(resolve, ms)  
4 })
```



이 함수는 "아직 해결되지 않은 약속(Promise)"을 즉시 반환합니다. 그리고 내부 타이머가 끝나면 resolve()를 실행해 약속을 지킵니다.

⚙ 동작 메커니즘



1. Promise 생성 (Pending)

대기 중

wait(3000)을 호출하자마자 "나중에 완료될게"라는 약속 객체가 만들어집니다. 아직은 완료되지 않았습니다.



2. 타이머 작동 (Timer)

진행 중

내부의 setTimeout이 실제로 시간을 쥅니다. Promise는 이 시간 동안 외부에서 '기다릴 수 있는' 상태가 됩니다.



3. Resolve 호출 (Fulfilled)

완료됨

시간이 되면 resolve()가 호출됩니다. 이제서야 Promise 상태가 Success로 바뀝니다.

</> 사용 예시

async-flow.js

```
1 console.log('A')
2 await wait(3000) // ⌚ 진짜 기다림
3 console.log('B')
```



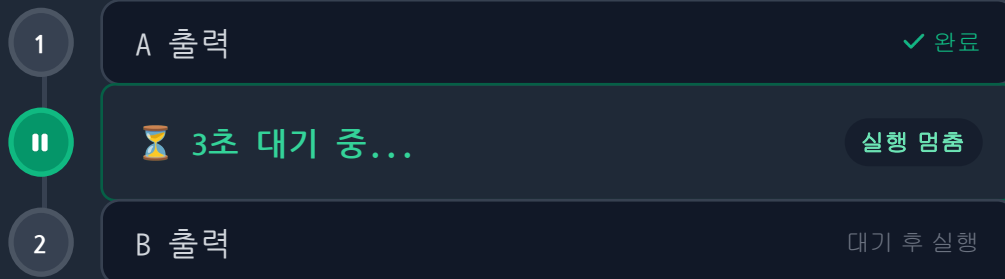
"자바스크립트야, wait(3000)이 끝날 때까지 여기에서 잠깐 멈추고 다음 코드 실행하지 마."

✔ 동작 방식

코드 실행을 일시 정지(Pause) 시킵니다.

- ▶ await 키워드를 만나면 Promise가 해결될 때까지 멈춥니다.
- ▶ 시간이 지나고 resolve되면 다시 실행을 재개합니다.

≡ 실제 실행 흐름



⚠ 주의: await는 async function 내부 또는 모듈 최상위(Top-level)에서만 사용할 수 있습니다.

LEGACY WAY

setTimeout()



```
setTimeout(callback, ms)
```

목적 실행을 단순 '예약'하고 넘어감

흐름 제어 논블로킹(Non-blocking), 다음 줄 즉시 실행

반환값 타이머 ID (숫자) - 취소용

에러 처리 콜백 내부에서 별도 try/catch 필요

Context: Global

비동기 예약

MODERN PATTERN

await wait()



```
await wait(ms)
```

목적 지정한 시간 동안 '실제로 기다림'

흐름 제어 해당 지점에서 일시 중단(Pause) 후 재개

반환값 Promise 결과 (void)

에러 처리 try/catch로 동기 코드처럼 처리가능

Context: async function

추천 패턴 ✓

</> 패턴 비교

final_check.js

1 // ❌ 나쁜 예: 예약만 하고 지나침

2 setTimeout(() => {}, 3000)

DON'T

3 console.log("즉시 실행됨")

4 // ✅ 좋은 예: 진짜 기다림 (Pause)

5 await wait(3000)

DO THIS

6 console.log("3초 뒤 실행됨")



'기다림'이 필요하다면 반드시 await 키워드와 함께 Promise를 반환하는 함수를 사용해야 합니다.



setTimeout() 은 그냥 예약

"3초 뒤에 이거 실행해줘"라고 던져두고, 기다리지 않고 다음 줄로 확 지나가버립니다.



await wait() 는 진짜 멈춤

"약속(Promise)이 끝날 때까지 여기서 꼼짝 말고 기다려"라고 명령합니다.

> 실제 코드 진행이 해당 라인에서 일시 정지됨

- 1 `setTimeout`은 기다리지 않는다. 단지 콜백을 예약할 뿐이다.
- 2 그래서 `setTimeout` 뒤에 있는 코드는 즉시 실행된다.
- 3 기다리려면 "기다렸다가 끝나는" Promise가 필요하다.
- 4 `wait(ms)`는 `ms` 후에 resolve되는 Promise를 만드는 함수다.
- 5 `await wait(ms)`는 진짜로 `ms` 동안 기다린 후 다음 코드를 실행한다.



Async Mastery

이제 `setTimeout`의 함정에 빠지지 않고,
Promise와 `await`로 실행 순서를
완벽하게 제어할 수 있습니다.

JS ES6+

Node.js